FINAL-YEAR PROJECT · INTERVIEW BRIEF

SeatWise

A multi-city restaurant reservation marketplace that also tells the owner which tables are about to go empty — and what to do about it.

> **One sentence.** Diners search and book across six Indian cities. Restaurant owners get a no-show risk score, a safe overbooking recommendation, and a waitlist seating plan — the same idea airlines use, applied to a dining room.

Stack: React 18 + Vite, Express, Prisma, PostgreSQL 16 (pgvector), optional Redis, JWT auth, OpenAI RAG concierge.

Demo login (no database needed):

- Diner — `diner@seatwise.app` / `demo1234`
- Owner — `owner@seatwise.app` / `demo1234` (opens the console)

Run: `npm install` then `npm run dev:web` → http://localhost:5173

1. The real-life problem

A table that is reserved but empty at 8:15 pm is not a small inconvenience. The kitchen bought for that cover, the server is standing, and the walk-in at the door is turned away. Indian restaurants commonly see a **10–20% no-show rate**. On a roughly ₹3,700 average check, that is hundreds of rupees per empty seat, against typical restaurant net margins of about 3–5%.

Example. Olive & Grove in Koregaon Park has 40 covers at 8 pm. Eight parties do not come. That is about ₹30,000 of food and labour planned for people who never sat down — and the family waiting outside was told "we are fully booked."

Existing booking sites solve the diner's problem (find a table). They barely solve the owner's problem (protect the table). SeatWise does both.

2. What the product is

SeatWise is two products in one app:

1. **Marketplace for diners** — search 34 restaurants across Pune, Mumbai, Bengaluru, Hyderabad, Chennai, and Delhi NCR. Filter by cuisine, neighbourhood, price band (₹ to ₹₹₹₹), party size, and "available tonight." Book a slot, join a waitlist, or ask in plain English.
2. **Yield console for owners** — tonight's book, a ranked no-show risk queue with reasons, an overbooking plan, floor and waitlist assignment, and 30-day analytics.

There are two ways to run it. **Demo mode** (default) uses generated data so a reviewer can click everything without Postgres. **Live API mode** uses Express, PostgreSQL, JWT, and optional OpenAI. Screens call the same service functions either way.

3. Interview pitch

30 seconds

> "SeatWise is a restaurant booking platform I built after auditing a PHP prototype that scored 2.5 out of 10 — SQL injection, XSS, and two databases that could not even join a guest to a reservation. I rebuilt it in React, Express, and PostgreSQL. Diners book across six cities. Owners see which bookings are likely to no-show, how many extra covers they can safely sell, and which waiting party should take the freed table. The risk model is a published logistic regression, so a host can see *why* a booking is 70% risk, not just a colour."

Two minutes (use this if they say "walk me through it")

1. **Problem.** A reserved empty table is lost revenue and a turned-away walk-in. Flat "overbook by 10%" is how you seat three parties of four into two tables and get a 1-star review.
2. **Diner flow.** Search Pune, party of 4, 8 pm, Mediterranean. Open Olive & Grove. See signature dishes with prices (Miso banana bread — ₹180). Hold the slot for five minutes. Confirm. Get a reference like SW-4KQ7MB.
3. **Owner flow.** Sign in as owner. Tonight's book shows Aditya Sharma, party of 2, 52% high risk — booked far ahead, unconfirmed, first-timer, Saturday prime time. Action: call or send a reminder.

Confirmation is the strongest protective signal in the model.

4. **Overbooking.** I do not sum probabilities and round down. Each booking is its own Bernoulli trial. The number of no-shows is a Poisson-binomial. I sell the largest extra covers that keep the chance of turning someone away under 5%.
5. **Engineering.** Double-booking uses `SELECT … FOR UPDATE` plus a Postgres exclusion constraint. Host edits (seated / no-show) use optimistic version checks. Allergen "no peanuts" is a SQL filter, not an LLM guess. Refresh tokens are hashed, httpOnly, and rotated.

4. Architecture (say this out loud)

Browser (React/Vite) → Vite `/api` proxy → Express (routes → controllers → services → Prisma) → PostgreSQL with pgvector. Redis is optional for cache and rate limits; the API still runs if Redis is down. OpenAI is used only for embeddings (`text-embedding-3-small`) and chat (`gpt-4o-mini`).

Money is stored in **paise** (integer), never floats, so a bill cannot become ₹1,249.999. Prices on screen use Indian grouping: ₹1,200.

5. Features, one by one

5.1 Discover and search

What it does. City switcher, text search, cuisine, area, price band, party size, time window (±90 minutes — "8 pm" means dinner, not only the exact slot), and quick filters: tonight, walk-in, counter, outdoor, group.

Real example. "Italian in Koregaon Park for 2 at 8." Forno Nove stays. Kite & String (only 2-tops, Kalyani Nagar, small plates) drops if the party is 6, because two joined tables still cannot seat them honestly. The card says when tables must be combined instead of pretending a six-top exists.

Ranking. Exact time match, then closeness, then rating, then editorial "hard to get," then distance. No paid placement.

5.2 Venue page and signature prices

Each restaurant has a tagline, about text, zones (main room, outdoor, counter, private), slots, and signature dishes with rupee prices. A ₹ café's dishes are cheaper than a ₹₹₹₹ omakase. Live API joins those names to real `MenuItem.priceInPaise` rows.

Example. Sourdough Society, Koramangala: "Miso banana bread — ₹180", "Cold brew, house roast — ₹220", "Six-seat evening menu — ₹1,800".

5.3 Booking with a real hold

Choosing a slot opens a sheet: time, guests, seating preference, name, phone, special requests. A countdown (about five minutes) matches the hold the API takes on a table. If it expires, confirm is disabled — the urgency is real, not a fake banner.

Example. Book Curry Culture Lab, Monday 9 pm, 2 guests, Main Room, name and phone required. Prepaid tasting menus show the deposit and the cancellation window (48 hours) before you confirm.

Validation matches the server: empty name → "Tell the restaurant who to expect." Bad phone → "A reachable number, please."

5.4 Waitlist

If the room is full, the guest joins a queue. The quoted wait is computed with the *same* seating algorithm the host floor uses, so the guest and the host do not see different numbers.

Example. Party of 2, free 2-top exists → seated now, wait 0. Party of 6 behind two smaller parties, next 6-top turns in 25 minutes → position and quote come from that packing, not a random "45 minutes."

5.5 My bookings and login gates

`/bookings` and `/console` require login. A diner who opens `/console` is sent home. A logged-out user is sent to login and returned afterward. Demo sessions are labelled in the UI so a reviewer knows it is not fake production security.

5.6 Auth

Register, login, logout, forgot password, reset password. Live mode: short-lived JWT in memory (not localStorage — one XSS must not become permanent account takeover), refresh token as an httpOnly `SameSite=strict` cookie, stored only as a hash, rotated on every use. Logout deletes the row. Passwords are bcrypt. Every body is Zod-validated. Rate limits on login and AI.

Example. Access token expires in 15 minutes. Three API calls 401 at once share one refresh, because three parallel refreshes would invalidate each other and look like token theft.

5.7 Owner console — Tonight's book

List of today's reservations: time, guest, party, status, no-show risk, actions Seat / No-show / Cancel / Remind. "No-show rate so far" is blank (**—**) until at least five bookings are already Completed or No-show. That is deliberate: 1 no-show out of 1 booking would display as 100% and lie to the host.

Example on screen. 6 pm, Aditya Sharma, 2 guests, Confirmed, 52% High risk, Remind. Siddharth Rao 36% Watch. Pooja Desai 4% Low risk — leave her alone.

Tiles: bookings today, covers, high-risk count, and "exposure tonight" — expected rupees at risk if nobody calls those high-risk parties.

5.8 Risk queue

Same bookings, sorted by probability, with the top reason ("Booked 40 days ahead", "Not confirmed", "First-time guest"). Bands are operational, not statistical:

Band	Probability	What staff do
Low	under 20%	Do nothing
Watch	20–50%	Send a reminder
High	50% and above	Phone call, or hold a card

Scores are clamped between 2% and 92% so staff never treat a model as certainty.

5.9 The no-show model (know the formula)

`P(no-show) = sigmoid(bias + weights · features)`, bias ≈ −2.05. Features:

- **Lead time as log(1+days)** — today vs three weeks matters a lot; two months vs three months barely does. A linear "days" term would make a 90-day booking 90× riskier, which restaurants do not see.
- **Party size, negative weight** — bigger parties show up more (occasion, social cost). The *money* lost if they flake is handled in the cost model, not by inflating the probability.
- **Confirmation answered** — strongest protection (about −1.15 log-odds).
- **Prior no-shows** — strongest risk (+0.95 each).
- **Prior visits** — help, but cap at 12. Loyalty saturates.
- **First-time guest, weekend, prime time (7–9 pm)** — raise risk.
- **Prepaid or card hold** — lower risk. Money changes behaviour more than a reminder.

Worked examples the coefficients were set to match:

- Booking A — 3 months out, party of 2, unconfirmed, first-timer, Saturday 8 pm → about **70%**.
- Booking B — booked yesterday, party of 4, SMS confirmed, 10 clean visits → about **5%**.

> **Say this before they ask.** The *form* is real logistic regression. The *weights are hand-set* to those two scenarios, not trained on a restaurant's history. Swapping in fitted weights later changes constants only. I do not claim a trained ML model.

5.10 Overbooking

Airlines sell 110 seats on a 100-seat plane because they know the *distribution* of no-shows, not because 10 feels lucky.

Counter-example you should tell. Two bookings, each 40% likely to no-show. Expected no-shows = 0.8. `floor(0.8) = 0` looks safe, but there is a 36% chance *nobody* no-shows. If you had sold one extra cover on that logic on a full room, you turn a guest away. SeatWise computes the exact Poisson-binomial (dynamic programming, fine at <40 bookings a slot) and only recommends extra covers while P(all of them show and you have no seat) stays under **5%**.

The API's simpler version is `floor(sum of probabilities)`, cached ~20 seconds. The console version is the stricter distribution. If asked which is "the" algorithm, say the console is the one you defend; the API slice is the cheaper approximation.

5.11 Floor and waitlist assignment

Not "first empty table." Largest party first (they have the fewest options), then the smallest table that fits, same zone only. If nothing fits, two tables in the *same* zone can be combined and the UI says so — someone must move furniture. A swap pass then reduces wasted seats.

Example. Free tables: one 2-top and one 6-top. Walk-in of 3. The 2-top is too small. Putting them on the 6-top wastes 3 seats a later party of 4 needed. The algorithm prefers the tightest legal fit across the whole queue, and will not seat a terrace booking indoors.

5.12 Analytics and the cost calculator

30-day occupancy, a day×hour heatmap of no-show rate, and slices by lead time, party size, and whether the guest confirmed. `/for-restaurants` lets you change covers and no-show percent and watch lost revenue move. That page is the sales pitch to a restaurant, using the same formula as the console.

5.13 Ask AI concierge

Live mode (RAG): question → embedding → pgvector cosine search, top 5 restaurants → gpt-4o-mini may recommend *only* from that retrieved context → reply plus venue cards. If embeddings were never generated, the API says so instead of inventing restaurants.

Demo mode: a local intent parser (budget, cuisine, area, party, tonight, outdoor). It is not pretending to be GPT. Say that clearly.

Example. "Date night under ₹1500, outdoor, Koregaon Park." Retrieved venues only. Peanut allergy is *not* decided by the model — the menu query excludes dishes whose allergen tags overlap, in SQL.

5.14 Menu, money, and owner edits

Menu items live in Postgres: name, description, price in paise, category, allergens, dietary tags, availability. Owners can create and update prices through the menu API. Allergen exclusion is `NOT allergens hasSome [...]`.

6. Why the rebuild exists

The `legacy/` folder is the original PHP prototype (audit 2.5/10), kept only for before/after comparison. Defects: SQL injection, XSS, split databases so a user row could not be related to a reservation. The Node app puts users, restaurants, tables, and reservations in one schema, validates every input, and escapes through Prisma.

7. Design decisions they will poke

Decision	Why
Pessimistic lock on booking	Two guests want the last table. `SELECT … FOR UPDATE` on candidate tables, overlap check, then a Postgres exclusion constraint if the app is wrong. Scoped by restaurant, so Pune and Mumbai do not block each other. Second request gets 409.
Optimistic lock on host status	Two hosts tapping "seated" is an accident, not a race for a scarce seat. `UPDATE … WHERE version = n`. Mixing the two on purpose is the point.
Allergens in SQL	"No peanuts" is a constraint. An LLM is slow and can be wrong. The concierge may talk about food; it does not clear a dish.
Refresh token rotation	Stolen refresh token dies on next legitimate use. Access token never sits in localStorage.
Hand-set logistic weights	The UI needs rank + explanation. Fitted weights drop in later without rewriting the console.
Poisson-binomial, not floor(Σp)	Expected value hides the "everyone shows up" tail.
Demo mode separate from live API	A reviewer can judge the product without Docker. Live mode is the same screens hitting Express.

8. Honest limitations (say them first)

- No-show coefficients are published, not trained on production history.
- No error tracker or distributed tracing yet — logs only.
- RAG runs in the request path. Under heavy load it can time out; a queue would be the next step. There is an hourly cap.
- City-wide "who can seat 4 at 8 tonight?" search is still on demo availability, not one SQL query across every venue. Booking one chosen venue is real in live mode.
- Some owner yield panels are complete in the browser demo; the live API covers reservations, waitlist, host status, and a simpler overbooking number.

Owning these is stronger than being caught.

9. Cross questions and answers

Q. Is this machine learning?

It is the *model form* of logistic regression, evaluated in JavaScript, with coefficients set so two research scenarios land near 70% and 5%. I would retrain on a venue's own completed vs no-show rows and replace the constants. I do not call it a trained model.

Q. Why not just use ChatGPT to predict no-shows?

A probability that changes every call cannot drive overbooking or a phone-call policy. Logistic regression is deterministic, fast, and explainable term by term. The LLM is only the concierge, and only over retrieved venues.

Q. Why is party size a negative coefficient? Bigger tables hurt more.

They hurt more in *rupees*, which is party size × check × probability, in the exposure tile. Empirically, large parties cancel less often because someone has coordinated the group. Putting that damage into the probability would double-count it.

Q. Walk through Booking A to ~70%.

Start at bias −2.05 (low base rate). Add log lead time for ~90 days, first-time guest +0.7, weekend +0.25, prime time +0.2. Do not subtract confirmation. Sigmoid of that sum, clamped, is about 0.7. Booking B subtracts a large confirmation weight and prior visits, so it collapses near 0.05.

Q. Why Poisson-binomial instead of a normal approximation?

A slot has tens of bookings, small probabilities, and the dangerous tail is "zero no-shows." Normal approximation is worst exactly there. DP is $O(n^2)$ and n is under 40.

Q. What if all three extra parties show up?

That probability is what the 5% tolerance caps. Above that, the recommendation is zero extra covers. The product would rather leave a table empty than bounce a guest.

Q. How do you stop double booking?

Transaction. Lock candidate tables with `FOR UPDATE`. Pick the smallest table that fits and does not overlap. Commit. A Postgres exclusion constraint on table + time range is the backstop if two requests slip through. Conflict → 409. Locks are per restaurant.

Q. Why not lock the whole restaurant row?

That would serialise every booking in the building, including tables that do not overlap. Locking only candidate tables keeps other zones bookable.

Q. Optimistic vs pessimistic — when?

Pessimistic when two customers compete for one scarce seat. Optimistic when two staff edit status; the loser retries. Using pessimistic everywhere would slow the floor. Using optimistic on the last table would lose bookings.

Q. Where is the JWT stored? Why not localStorage?

Access token is in a JS variable. Refresh token is an httpOnly cookie. localStorage is readable by any script; this codebase's history included XSS, so a stolen localStorage token would have been permanent.

Q. What is refresh rotation?

Each refresh issues a new random token, stores its hash, and invalidates the old one. Reuse of an old token is treated as theft and the session family can be killed. Logout deletes the hash, so "log out" actually ends the session.

Q. How do you prevent email enumeration on forgot-password?

The API always answers "if that email exists, a reset link is on its way," whether or not the account is real. Reset tokens are hashed, expire, and are single use.

Q. Why is allergen filtering not in the prompt?

A safety filter must be complete and repeatable. SQL `NOT hasSome` either excludes the dish or it does not. An LLM can miss a peanut in a sauce description. The assistant may mention dietary preferences; the menu query enforces them.

Q. Can the chatbot invent a restaurant?

In live RAG, no. The prompt is grounded on the top-5 pgvector hits. Empty index returns an explicit "generate embeddings" message. Demo mode uses the fixture list, also not free-form invention of venues outside the catalogue.

Q. Why pgvector?

The catalogue is small enough to embed per restaurant (name, area, vibe, menu, prices) and search by cosine similarity inside Postgres. No separate vector database to operate.

Q. What if Redis is down?

Cache and some rate limits degrade. Booking correctness does not depend on Redis; it depends on Postgres locks and the exclusion constraint.

Q. Why paise not rupees as a float?

Binary floats cannot represent 0.1 exactly. A confirmation screen should never show ₹1,249.999. Integer paise, format with `Intl` / `en-IN` at the edge.

Q. Why does no-show rate show a dash?

The tile is today's realised rate: no-shows divided by (completed + no-shows). Until five bookings have actually finished, the sample is too small. Confirmed future parties are predictions in the risk column, not in that percentage.

Q. How is search relevance not "whoever pays"?

Score is availability at the requested time, rating, a small editorial boost, and distance. There is no sponsored slot.

Q. Time zones?

A booking date is the venue's calendar date, not the browser's UTC conversion. India is fixed at UTC+330. That avoids "your table was yesterday" for anyone west of the restaurant.

Q. How would you test the double-book?

A concurrency test fires many parallel creates at one slot against real Postgres. Exactly one succeeds; the rest get a conflict. I can describe that test; I do not need to pretend I load-tested production traffic.

Q. What would you build next?

Fit the logistic weights on real outcomes. Add tracing so a host sees "risk queue empty because scoring failed." Put concierge calls on a queue. One cross-venue availability query. Finish yield writes on the live API so the console is not demo-only for overbooking math.

Q. Why a monolith API instead of microservices?

One team, one deploy, one database transaction that must lock tables and insert a reservation together. Splitting booking and inventory would make the double-book bug easier, not harder.

Q. GDPR / privacy of guest phone numbers?

Phone is collected because the venue may need to call a high-risk party. It is not shown with the risk score to other diners. Access tokens are not in localStorage. I would add retention limits and a delete-account path before a real launch. I have not claimed production compliance.

Q. What was the hardest bug?

The booking sheet could not accept typing. A five-minute countdown re-rendered every 250 ms, which recreated `onClose`, which re-ran the dialog focus effect and stole focus from the inputs. Fix: keep `onClose` in a ref and run the focus effect only when the sheet opens.

Q. Show me you understand the legacy audit.

PHP app, score 2.5/10. SQL built by string concatenation, XSS in rendered fields, and reservations in a different database from users — so you could not ask "what did this guest book?". The new schema is one Postgres database, Prisma parameterised queries, and React text nodes instead of raw HTML injection.

10. Demo script (4 minutes)

1. Open Discover. Switch city to Pune. Filter Mediterranean or search "olive." Open Olive & Grove. Point at signature prices.
2. Pick 8 pm, party of 2, confirm a booking. Show the reference.
3. Log out. Open `/bookings` — redirected to login. That is the guard.
4. Sign in as `owner@seatwise.app` / `demo1234`. Land on the console.
5. Tonight's book: point at a high-risk row and the Remind action. Explain the dash on no-show rate.
6. Risk queue: read the top driver out loud.
7. Overbooking: "extra covers only while walk-away risk stays under 5%."
8. Floor: largest party, tightest table, same zone.
9. Optional: `/for-restaurants` calculator. Optional: Ask AI with "outdoor seating in Koregaon Park."

11. Numbers worth memorising

Item	Value
Cities / venues in the demo market	6 cities, 34 restaurants
High-risk threshold	≥ 50% → call; ≥ 20% → remind
Overbooking tolerance	P(turning someone away) < 5%
Score clamp	2% to 92%
Access token	about 15 minutes, memory only
Refresh token	about 7 days, hashed, rotated, httpOnly
Slot hold	about 5 minutes
Worked risks	~70% far-out unconfirmed; ~5% confirmed regular
Industry pain	10–20% no-shows; ~3–5% net margin
Legacy audit	2.5 / 10 PHP prototype

SeatWise interview guide. Figures match the code in `apps/web/src/lib/risk.js`, `overbooking.js`, `assignment.js`, and the README design-decisions section. Demo data is generated; live mode needs Postgres, migrations, and an OpenAI key only for the concierge.